

PRD — Assisted Commerce 2.0: Ring AI Voice-Bot Integration (Phase 1)

Document type

Author

Status

Last updated

Reviewers

Related docs

How to read this document. Sections 1–6 define *what* we are building and *why*. Section 7 defines the *integration contract* with Ring AI at engineering altitude (field mappings, webhook contract, lead state machine) without prescribing our internal database schema — that is deliberately left to engineering discovery. Section 11 holds **all open questions, split into two blocks**: one for Truemed's engineering and one for the Ring AI tech team. Anything marked **[DISCOVERY]** requires an engineering answer before estimation; anything marked **[RINGG]** requires a written answer from the vendor.

1. TL;DR

Replace the top of the Assisted Commerce (ACOM) funnel — today a 100% human outbound-calling layer on incomplete/abandoned orders — with a Ring AI voice bot that calls eligible leads, qualifies intent (**Hot / Warm / Cold**), and hands only Hot/Warm leads to human agents for order placement. Humans stay in the loop for the actual order in Phase 1.

A POC across ~1,300 real leads showed **~20% lead-to-sale conversion on bot-transferred leads vs ~5% on the human-only baseline**, implying roughly **75% lower manpower** for the same output, while bot concurrency lets us finally cover the ~40% of eligible leads we cannot attempt today. This PRD scopes the production integration: how Ring pulls orders from us, calls and self-manages retries/scheduling, and returns verdicts + conversation context to us via webhook, plus the portal changes human agents need.

2. Problem & Strategic Context

2.1 The problem

The ACOM team manually cold-calls every eligible incomplete order. This is capacity-limited and expensive:

- **Coverage gap:** Of ~17,000 eligible leads per month (those with both patient details *and* address on file), we attempt only ~10,000 — **~40% are never called**, a standing pool of unconverted revenue.
- **Low conversion:** Lead-assigned → lead-converted is **~5%**.
- **Cost:** Current CPO ≈ ₹250, on a total cost (payroll + variable telephony/ops) of ~₹50L against ~20,000 orders/month.
- **Operational drag:** Slow time-to-first-call, agent bandwidth wasted on cold/dead leads, day-on-day spillover, and poor line connectivity on the current telephony provider (Knowlarity packet-loss issues).

2.2 Why now

The Ring AI POC demonstrated a 4x lift on transferred-lead conversion (~20% vs ~5%) and the ability to scale outreach via concurrency. This both **increases revenue** (faster, broader outreach across the full 17k universe) and **reduces variable cost** (fewer human agents needed per converted order). Commercials (per-call vs outcome-based) are still being negotiated and will be appended when finalised.

2.3 Baseline vs proposed funnel

Today (baseline): 100% human-driven. All eligible leads called by agents → ~5% convert. Capacity-capped at ~10k/17k.

Proposed (Phase 1): AI bot calls customer → intent identified (Hot / Warm / Cold) → Hot/Warm leads assigned to human agents with target callback **< 5 min** → human places order. Cold and unconnected leads are handled per the rules in §6.

3. Goals, Non-Goals & Success Metrics

3.1 Goals (Phase 1)

1. Stand up a production integration where Ring AI pulls eligible orders, places qualifying calls, self-manages retries/scheduling, and returns a verdict + context for each order.
2. Expand attempted coverage from ~10k toward the full ~17k eligible universe per month.
3. Preserve and improve the ~20% transferred-lead conversion seen in the POC.
4. Reduce CPO meaningfully vs the ~₹250 baseline (target to be set once commercials close).
5. Give human agents everything they need (bot summary, objection/reason tags, recording/transcript) to convert a transferred lead quickly.

3.2 Non-Goals (Phase 1)

- The bot does **not** place orders, edit carts, capture/modify address, apply coupons, or perform SKU search. (That is Phase 2 — see §9.)
- No change to the core order-assigning logic itself beyond adding an eligibility filter for Ring (§6.3).

- Empty-cart handling logic on our side — handled at the bot's end in Phase 1.

3.3 Success metrics

Metric

Transferred-lead → sale conversion

Eligible-lead coverage (attempted/eligible)

CPO

Human callback latency on Hot/Warm

Bot call-connect rate

Intent-label accuracy (Hot/Warm precision)

[DISCOVERY] Confirm the exact metric definitions and event sources we will instrument so dashboards are built once (see \$10 and Open Questions).

4. Users & Personas

- **Incomplete-order customer** — added items to cart, did not place the order within 30 min of cart inactivity. Receives the bot call.
 - **Category 1 human agent** — handles Hot/Warm leads transferred by the bot; places the order. Needs bot context up front.
 - **Category 2 human agent** — handles leads *not* covered by Ring (overflow / non-eligible) via the existing manual flow, unchanged.
 - **ACOM Ops lead** — controls Ring concurrency, monitors funnel health, owns the fallback switch.
 - **Truemeds Engineering** — builds the integration, the assigning-logic filter, webhook ingestion, and portal changes.
 - **Ring AI Tech Team** — owns the bot, calling infra, retry/schedule engine, and webhook emission.
-

5. Solution Overview

We treat **Ring as a virtual agent pool** that "pulls" orders exactly as a human agent does when clicking *Assign Order*, governed by the existing assigning logic plus one extra eligibility filter. Ring takes ownership of an order, calls and self-manages all intermediate states (retry, schedule, max attempts), and **closes the thread** by returning one of a small set of terminal outcomes via webhook. We ingest those webhooks, route Hot/Warm leads to Category 1 agents through the existing assigning logic, and surface bot context in the portal.

Two human queues run in parallel:

- **Category 1** — Hot/Warm leads from the bot.
- **Category 2** — leads Ring did not handle (overflow when concurrency < inflow, or non-eligible orders), running the current manual flow untouched.

A single control — **Ring concurrency** — scales the bot layer up or down, and dropping it to **0** is the fallback that reverts everything to today's all-human flow.

6. Functional Requirements & Process Flow (Phase 1)

6.1 Trigger / eligibility entry

An order enters the ACOM waiting pool when it has **>30 min of inactivity after a cart modification without the order being placed** (existing rule, unchanged).

6.2 Order ownership model

- Ring pulls an order from us via the assigning logic, the same way a human agent does on *Assign Order*. Concurrency defines how many orders Ring may hold at once and how fast it keeps pulling.
- While Ring owns an order it is **removed from the ACOM human pool**.
- The order returns to us **only when Ring closes the thread** (terminal outcome below).

6.3 Ring eligibility filter (added layer on top of existing assigning logic)

Only send an order to Ring when **all** are true:

1. Patient details are filled — a valid **patient_id** exists for the order.
2. Address details are filled — a valid **address_id** exists for the order.
3. (Empty-cart cases are handled at the bot's end, not filtered out here.)

All other ordering within the assigning logic (primarily **AOV descending**) is unchanged.

[**DISCOVERY**] Where do `patient_id` and `address_id` live, and can the assigning service cheaply assert their presence at assignment time? This is a primary discovery item — see §11.A.

6.4 Bot-side lifecycle (Ring-owned) and terminal outcomes

Ring owns every intermediate state until the thread closes. Terminal outcomes returned to us:

#

1

2

—

Retry logic (count, delay on busy/not-picked/failed) and scheduling are configured on the call (`call_config.call_retry_config`, `call_config.call_time`) — see §7.

6.5 Human routing (Category 1)

On receiving a Hot/Warm webhook, we filter the lead and assign it to a Category 1 agent **via the current assigning logic (AOV-driven)**. Existing hold/re-attempt/schedule logic for human agents stays the same. Target: agent callback < 5 min.

6.6 Cold, unconnected & overflow handling

- **Cold leads** and **max-attempt-no-connect** orders return to the pool. Phase-1 default: treat per current policy. **(P1 enhancement, BRD)**: attempt Cold leads on a priority logic only when the Hot/Warm queue is empty / in max-attempts / on hold — refined later via quality feedback at the bot's end.
- **Overflow**: when concurrency < order inflow, residual eligible orders flow to **Category 2** (current manual flow).
- **Edge case (low inflow, many Category-1 agents idle)**: spin up a *replica Category 2* so idle agents take normal-flow orders. Rare; flagged for the solve.

6.7 Fallback / kill-switch

If anything fails, **set Ring concurrency to 0**. In-flight Ring-owned threads complete or time out, no new orders are pulled by Ring, and the funnel reverts to the all-human flow. Concurrency and inter-attempt timing are controllable on our side.

6.8 Telephony note

Ring recommends low-latency / low-packet-loss trunks (Knowlarity/TATA Tele). ACOM currently uses Knowlarity, which has had packet-loss issues. Switching to **TATA** is expected to be a **single number/config change in our backend**. To be evaluated; tracked as a dependency, not a blocker.

6.9 Compliance & recording

NDA with Ring is signed. **Call recordings must be stored on our end** for every bot conversation, alongside the transcript link. Retention and PII handling per Truemeds policy. (Confirm storage location/retention in discovery.)

7. Integration Contract — Ring AI API (engineering-ready)

This section maps our requirements onto the Ring AI API. It defines the *contract*, not our internal storage design.

7.1 Endpoint

- **Initiate call**: POST /calling/outbound/individual on base https://prod-api.ringg.ai/ca/api/v0.
- **Auth**: header X-API-KEY: <key> (keep server-side; never client-exposed).
- **Note — deprecation**: Ring marks this endpoint *deprecated* in favour of the **pool-based v2** (initiate-individual-call-v2), which auto-resolves caller numbers (rotation,

spam-skipping, auto-purchase). **[RINGG]** we should confirm which endpoint to build against for production (see §11.B). The field model below is largely shared.

7.2 Required request fields

Field

name

mobile_number

agent_id

from_number_id **or** from_number

7.3 Custom variables (custom_args_values) — the personalisation payload

Referenced in the bot prompt as @{{variable_name}}. Keys must stay **stable** across our code, Ring prompts, and analytics. POC used a thin set; expanding it is the single biggest conversion lever.

POC variables (carry forward):

- Customer: callee_name, mobile_number
- Order: order_id, cart contents (SKU + Qty), total_mrp, total_sp, total_discount_pct, previously bought medicines
- Address (to be sent as **5 discrete fields**, not one blob): flat, building, area, city, state, pincode
- ETA: eta_days

Phase-1 expansion candidates (to validate from call recordings):

- tm_cash_balance (TM Cash available to the customer)
- Additional objection-relevant flags (TBD from recording analysis)

Data-format requirements:

- Address sent as discrete fields so the bot can answer contextually — e.g. on an ETA question say "*eta_days days, area, city*"; on a full-address question read the complete address. **[RINGG]** confirm whether Ring can delimit/segment a structured field on their side, or whether we must always pre-split. (§11.B)
- **No "NA"/null leakage** into the prompt. Engineering must implement **validation/sanitisation**: drop or substitute missing fields gracefully so the bot never voices "NA". **[DISCOVERY]** define the validation rules and fallbacks per field. (§11.A)

7.4 Call configuration (call_config)

Setting

Retry attempts

Retry delays (min)

Calling window

Scheduled call

Idle timeouts
Max call length

Decision needed: are retry/schedule/window managed **per-call by us** (passed in each request) or **configured once on the Ring assistant/campaign?** [RINGG] (§11.B)

7.5 Optional fields we likely want

- `smart_formatter` — `extract_first_name` + optional transliteration (en→hi) to humanise `callee_name`; preserves `original_callee_name`. Evaluate for Hindi/Hinglish calls.
- `call_category` — internal grouping/reporting (e.g. `acom_incomplete_order`).
- `version_id` — pin a specific assistant version for consistency across a campaign.
- `parent_call_id` — for chained/follow-up flows (future).
- `callback_url` / `callback_args` — per-request webhook target + headers/params (see §7.6).

7.6 Webhook / callback contract (Ring → Truemedics) — critical path

On call initiation Ring returns `data.call_id` (plus `call_status`, `initiated_at`, `agent_id`, echoed `custom_args_values`). **We must store `call_id` and map it to our `order_id`.**

We need Ring to call our webhook on **terminal events** carrying the **structured data the BRD requires**:

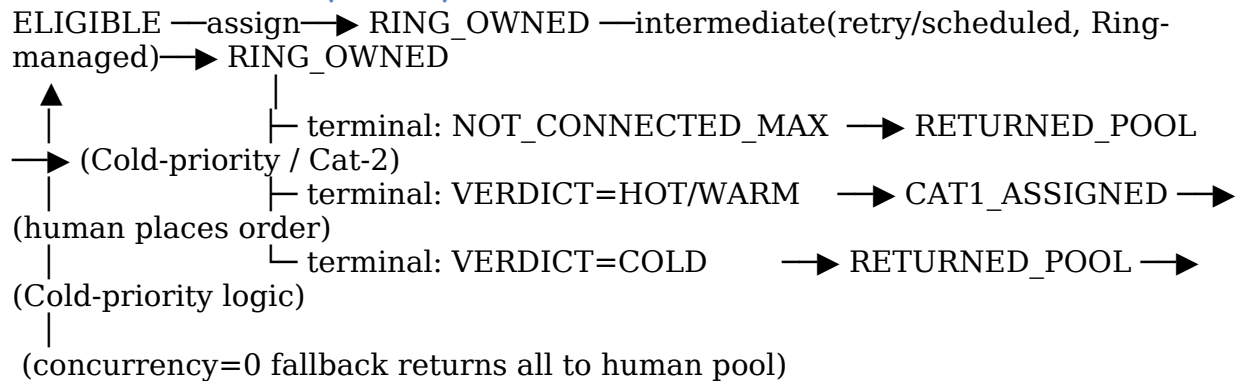
- **Intent label** — Hot / Warm / Cold
- **Conversation context / summary**
- **Reason / objection tag**
- **Transcript link** and **recording link**
- **Identifiers** — `order_id`, `mobile_number`, `call_id` (to reconcile against our record)
- **Terminal call status** — from Ring's enum: completed, failed, cancelled, forwarded, plus registered/ongoing/retry/error as intermediate.

[RINGG] open items on the webhook (see §11.B):

1. Exact webhook **event payload schema** for terminal events (the docs cover initiation + retrieval, not the push schema for verdicts).
2. Are intent label, objection tag, summary, transcript & recording URLs **all delivered in the webhook**, or must we pull them via `GET /history/get-call-details`?
3. Webhook **auth/signature** scheme (so we can verify authenticity) and retry behaviour on our 5xx.
4. Is the webhook configured **per-request** (`callback_url`) or **globally** on the workspace?

Reconciliation endpoints available as backstops: `GET /history/get-call-details` and `GET /history/get-call-history`.

7.7 Lead state machine (our side)



A thread is **open** from RING_OWNED until a terminal webhook; no human action occurs on an order while it is RING_OWNED.

7.8 Error / failure handling (from API)

Symptom

401 Unauthorized

400 validation error

Call stuck queued

Assistant says wrong data

8. Portal Changes (human-facing)

1. **Pre-populated context box** on the order-detail page: bot summary (intent reason, objections raised).
2. **Recording / transcript link** for a quick recap before the agent dials back.
3. **Human-agent dispositions (P1)**: capture, via dropdown + free text, the reason an order was *not* confirmed — feeds back to tighten Hot/Warm classification rules. This is the quality loop that keeps the funnel high-precision.

[DISCOVERY] Portal data model: where do bot summary / tags / links attach to the order record, and how does the order-detail page render them? See §11.A.

9. Phase 2 — Fully Agentic Flow (outline only)

Once models prove sustained accuracy, extend agentic control so the bot itself performs actions instead of only qualifying: **cart edits, address capture/edit, coupon application, SKU search, and ultimately order placement** — removing the human from order placement for a growing share of leads. This requires deeper, write-path integration (transactional APIs, idempotency, payment/coupon rules, stronger guardrails and audit) and is explicitly **out of scope for this PRD**. It is captured here only to ensure Phase-1 design choices (stable identifiers, structured variables, disposition feedback loop) don't block Phase 2.

10. Analytics & Instrumentation

Track the funnel end-to-end: eligible → sent-to-Ring → connected → verdict (Hot/Warm/Cold) → Cat-1 assigned → converted, plus connect rate, attempts-to-connect, callback latency, CPO, and intent-precision via dispositions. Build dashboards once metric definitions and event sources are locked (§3.3, §11.A). Cost/CPO structures and the directional CPO-reduction model are WIP pending commercials and will be appended.

11. Open Questions

11.A — For Truemedics Engineering (internal) — [DISCOVERY]

These gate estimation. Each is a "where/how do we store or assert this" question.

1. **Identifier storage & assertion.** Where do patient_id and address_id live, and can the assigning service assert their presence cheaply at assignment time (to power the §6.3 eligibility filter)?
2. **Variable sourcing.** For each custom_args_values field (cart SKUs+qty, MRP/SP/discount, previously bought meds, 5-part address, eta_days, tm_cash_balance), what is the system of record and can we fetch all of it at call-initiation time within latency budget?
3. **Data validation/sanitisation.** What are the per-field rules to prevent "NA"/null leakage into the prompt, and the fallback values when a field is missing?
4. **Order ownership lock.** How do we mark an order RING_OWNED so it's removed from the human pool, and reliably release it on a terminal webhook (including crash/timeout safety)?
5. **Webhook ingestion.** Where does the inbound Ring webhook land (service/endpoint), how do we map call_id→order_id, and how do we make ingestion idempotent + retry-safe?
6. **Portal data model.** Where do bot summary, objection/reason tag, intent label, transcript & recording links attach to the order, and how are they rendered on the order-detail page?
7. **Recording storage.** Where/how long do we store recordings on our side (bucket, retention, PII/compliance), and do we store Ring's link or pull/copy the file?
8. **Concurrency control.** How is the Ring concurrency value implemented and changed at runtime (including the →0 kill-switch), and who can change it?
9. **Cold-priority logic (P1).** Where does the priority queue for Cold re-attempts live and how does it interact with the AOV-driven assigning logic?
10. **Metric instrumentation.** Which events/tables source each success metric in §3.3 so dashboards are built once?
11. **Telephony switch.** Confirm the Knowlarity→TATA change is a single backend config and scope the change/testing.

11.B — For Ring AI Tech Team (vendor) — [RINGG]

These gate the integration contract.

1. **Endpoint choice.** Should production build against the deprecated initiate-individual-call or the pool-based **v2** (initiate-individual-call-v2)? What changes in the request/response between them?
 2. **Webhook payload schema.** Provide the exact schema for terminal/verdict webhook events — fields, types, and examples.
 3. **Verdict delivery.** Are intent label (Hot/Warm/Cold), objection/reason tag, conversation summary, transcript URL and recording URL all pushed in the webhook, or must some be pulled via GET /history/get-call-details?
 4. **Webhook auth & reliability.** What signature/auth scheme secures the webhook, and what is the retry policy if our endpoint returns 5xx?
 5. **Callback configuration.** Is callback_url set per-request or globally per workspace? Recommended pattern for our volume?
 6. **Retry/schedule/window ownership.** Are retry count/delays, calling window, and scheduling set per-call (in each request) or once on the assistant/campaign? What are the defaults we'd inherit?
 7. **Structured fields (e.g. Address).** Can Ring segment/delimit a structured field on their side, or must we always pre-split? Best practice for the 5-part address use case.
 8. **Agent/version pinning.** How do we obtain and pin agent_id + version_id for a stable production assistant, and how are prompt/version updates rolled out?
 9. **Caller IDs.** How do we obtain from_number_id(s); do they support number rotation/pooling for deliverability and spam-avoidance at our volume?
 10. **Concurrency & rate limits.** What concurrency/throughput can Ring sustain for us, what are API rate limits, and how do we raise/lower concurrency programmatically?
 11. **Credits/health signals.** How do we monitor workspace credits and provider health to avoid silent queued stalls?
 12. **Telephony.** Confirmation that low-latency TATA trunks are supported/recommended and any config needed on Ring's side.
-

12. Dependencies, Risks & Assumptions

Dependencies: Ring assistant + caller numbers provisioned; webhook schema delivered; assigning-logic filter built; portal changes; recording storage; commercials finalised; telephony decision (Knowlarity vs TATA).

Risks:

- *Intent-precision drift* — bad Hot/Warm labels waste agent time. Mitigation: disposition feedback loop (§8.3), ongoing quality control.
- *Deprecated endpoint* — rework if we build on v1. Mitigation: resolve §11.B.1 before build.

- *Data quality* (NA/null leakage, stale variables) → poor conversations. Mitigation: validation layer (§11.A.3).
- *Webhook reliability* → lost verdicts / stuck orders. Mitigation: idempotent ingestion + reconciliation via history endpoints (§7.6).
- *Telephony packet loss* → poor call quality. Mitigation: evaluate TATA.

Assumptions: POC ~20% conversion is reproducible at scale; ~17k eligible universe is stable; NDA covers production recording storage; commercials won't change the core flow.

13. Phasing / Rollout (proposed)

1. **Discovery** — answer §11.A and §11.B; lock endpoint and webhook schema.
 2. **Build** — eligibility filter, call-initiation service, webhook ingestion + state machine, portal context box + dispositions, recording storage.
 3. **Controlled pilot** — low concurrency on a slice of eligible orders; verify verdict accuracy and callback latency; tune retry/window/variables.
 4. **Scale** — ramp concurrency toward full eligible universe; monitor metrics; keep the concurrency=0 kill-switch ready.
 5. **Phase 2 evaluation** — only after sustained accuracy.
-

End of draft v0.1. Sections 7 and 11 are the engineering-discovery surface; §11.B is the vendor-question set to send Ring AI.